Jenkins Practical — Complete macOS Roadmap

Your teacher's material is written mainly for **Windows**, so I would **not follow it literally on your Mac**. I'll keep the same tasks, job names, concepts, and expected results, but translate the commands and installation steps to macOS.

The overall order is:

macOS prerequisites → Java → Jenkins → Jenkins initial setup → Git/GitHub → Jenkins plugins → Freestyle basics → Pipeline basics → Git integration → Poll SCM → Webhooks → Build numbers → Artifacts → Security → Agents/Executors → final evidence

---

PART 0 — Understand what you are going to build

By the end, you should have these Jenkins jobs.

Freestyle jobs

Order	Jenkins Job
1	`HelloWorldJob`
2	`WorkspaceJob`
3	`FileCopyJob`
4	`ParameterizedJob`
5	`ScheduledJob`
6	`GitJob`

These correspond directly to your teacher's Freestyle practical.

Pipeline jobs

Order	Jenkins Job
7	`EchoPipeline`
8	`BuildTestPipeline`
9	`FilePipeline`
10	`UserPipeline`
11	`ScheduledPipeline`
12	`GitPipeline`

These are the six Pipeline exercises in your teacher's Pipeline document.

Automatic-build jobs

Order	Jenkins Job
13	`Jenkins-Polling-Demo`
14	`Jenkins-Webhook-Demo`
15	`Jenkins-Pipeline-Demo`
16	`Jenkins-Version-Demo`

PART 1 — Install the macOS prerequisites

Your teacher allows:

- JDK 11
- JDK 17
- JDK 21

For your Mac, I recommend **JDK 21**.

Step 1 — Open Terminal

Press:

Command + Space

Type:

```
Terminal
```

Press Enter.

Step 2 — Check whether Homebrew is installed

Run:

```
brew --version
```

If you see something like:

```
Homebrew ...
```

you're good.

If you get:

```
command not found: brew
```

install Homebrew from its official site.

After installation, reopen Terminal.

Then:

```
brew --version
```

PART 2 — Install Java

Step 3 — Install JDK 21

```
brew install openjdk@21
```

Then check:

```
java -version
```

If Java is already installed and you see Java 21, don't reinstall it.

Step 4 — Configure Java on macOS

Run:

```
echo 'export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"' >> ~/.zshrc
```

Then:

```
source ~/.zshrc
```

Check:

```
java -version
```

Also:

```
which java
```

Step 5 — Set JAVA_HOME

Run:

```
echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 21)' >> ~/.zshrc
```

Then:

```
source ~/.zshrc
```

Check:

```
echo $JAVA_HOME
```

And:

```
java -version
```

Stop here and verify

You want:

```
java version "21..."
```

before proceeding.

PART 3 — Install Git

Check:

```
git --version
```

If Git isn't installed, run:

```
xcode-select --install
```

Then verify:

```
git --version
```

PART 4 — Configure your Git identity

Run:

```
git config --global user.name "Your Name"
```

Then:

```
git config --global user.email "your-email@example.com"
```

Check:

```
git config --global --list
```

PART 5 — Install Jenkins

For macOS, don't download the Windows installer mentioned in your teacher's document.

Use the macOS/Homebrew installation method.

Step 6 — Install Jenkins

```
brew install jenkins-lts
```

Then:

```
brew services start jenkins-lts
```

Check:

```
brew services list
```

You should see Jenkins running.

PART 6 — Open Jenkins

Open your browser.

Go to:

```
http://localhost:8080
```

PART 7 — Find the initial Jenkins password

First try:

```
cat ~/.jenkins/secrets/initialAdminPassword
```

If that doesn't work:

```
find ~/.jenkins -name initialAdminPassword 2>/dev/null
```

Copy the password shown by the command.

Do not guess the password.

PART 8 — Jenkins Initial Setup

The Jenkins wizard should appear.

You'll see:

Unlock Jenkins

Paste the initial administrator password.

Then choose:

Install suggested plugins

Let Jenkins finish installing the plugins.

PART 9 — Create your Jenkins administrator

Enter:

```
Username:
admin
```

Choose a strong password.

Then enter:

```
Full name:
Your Name

Email:
your-email@example.com
```

Click:

Save and Continue

Then:

Start using Jenkins

PART 10 — Verify Jenkins is working

You should now see:

Jenkins Dashboard

On the left you'll see things such as:

- New Item
- People
- Build History
- Manage Jenkins
- Credentials

At this point:

> **Jenkins installation is complete.**

PART 11 — Configure Git/GitHub support

Go to:

Manage Jenkins → Plugins

Check that Git-related plugins are installed.

Look for:

```
Git
Git client
GitHub
```

If required, install them from:

Available plugins

PART 12 — Understand Jenkins before doing practicals

CI

Continuous Integration means developers integrate changes frequently and the system automatically builds/tests them.

CD

Continuous Delivery means software is kept in a state where it can be released at any time.

SCM

Source Code Management.

For us:

```
Git + GitHub
```

Job

A task configured in Jenkins.

Build

One execution of a Jenkins job.

Workspace

The directory where Jenkins performs the job.

Pipeline

A Jenkins workflow consisting of stages.

Artifact

Something produced by a build.

Trigger

Something that causes Jenkins to start a build.

PART 13 — Your first Jenkins job

FREESTYLE TASK 1 — HelloWorldJob

Create:

```
HelloWorldJob
```

Select:

Freestyle project

Click:

OK

Add shell command

Scroll to:

Build Steps

Click:

Add build step → Execute shell

On macOS/Linux, use:

```
echo "Hello, Jenkins!"
```

Click:

Save

Then:

Build Now

Click the build number, for example:

```
#1
```

Then:

Console Output

You should see:

```
Hello, Jenkins!
```

Task complete.

FREESTYLE TASK 2 — WorkspaceJob

Create:

```
WorkspaceJob
```

Select:

Freestyle project

Build Step:

Execute shell

Use:

```
echo "The workspace path is: $WORKSPACE"
```

Save → Build Now → Console Output.

You should see something similar to:

```
The workspace path is: /Users/.../.jenkins/workspace/WorkspaceJob
```

FREESTYLE TASK 3 — FileCopyJob

The teacher's Windows version copies to:

```
C:\inetpub\wwwroot\Devops\
```

We won't use that on your Mac.

Instead, create a folder in your home directory.

Step 1 — Terminal

```
mkdir -p ~/jenkins-devops-copy
```

Step 2 — Jenkins job

Create:

```
FileCopyJob
```

Freestyle.

Build Step → Execute shell:

```
echo "Hello from Jenkins" > "$WORKSPACE/test.txt"

cp "$WORKSPACE/test.txt" "$HOME/jenkins-devops-copy/test.txt"

echo "File copied successfully."
```

Step 3 — Build

Click:

Build Now

Then:

Console Output

Verify:

```
File copied successfully.
```

Step 4 — Verify from Terminal

```
cat ~/jenkins-devops-copy/test.txt
```

Expected:

```
Hello from Jenkins
```

FREESTYLE TASK 4 — ParameterizedJob

Create:

```
ParameterizedJob
```

Select:

This project is parameterized

Click:

Add Parameter → String Parameter

Name:

```
USERNAME
```

Default value:

```
User
```

Build Step → Execute shell:

```
echo "Hello, $USERNAME"
```

Save.

Click:

Build with Parameters

Enter:

```
Timothy
```

Build.

Then try another value.

Console should show:

```
Hello, Timothy
```

FREESTYLE TASK 5 — ScheduledJob

Create:

```
ScheduledJob
```

Freestyle.

Go to:

Build Triggers

Select:

Build periodically

Enter:

```
H/2 * * * *
```

Build Step → Execute shell:

```
echo "Current date and time: $(date)"
```

Save.

Wait and verify multiple automatic builds.

After verifying it works, **disable the schedule** so it doesn't keep running forever.

FREESTYLE TASK 6 — GitJob

Create GitHub repository:

```
jenkins-git-demo
```

Add:

```
index.html
```

Then create Jenkins:

```
GitJob
```

Freestyle.

Go to:

Source Code Management → Git

Repository URL:

```
https://github.com/YOUR_USERNAME/jenkins-git-demo.git
```

Branch:

```
*/main
```

Build Step → Execute shell:

```
git log -1 --pretty=%B
```

Save → Build Now → Console Output.

You should see the latest commit message.

PART 14 — Pipeline tasks

Now move to Pipelines.

PIPELINE TASK 1 — EchoPipeline

Create:

```
EchoPipeline
```

Select:

Pipeline

Under Pipeline → Pipeline script:

```
pipeline {
    agent any

    stages {
        stage('Echo') {
            steps {
                echo 'Hello, Jenkins Pipeline!'
            }
        }
    }
}
```

Save → Build Now → Console Output.

Expected:

```
Hello, Jenkins Pipeline!
```

PIPELINE TASK 2 — BuildTestPipeline

Create:

```
BuildTestPipeline
```

Pipeline script:

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building the project...'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application...'
            }
        }
    }
}
```

Run it.

You should see:

```
Build
   ↓
Test
   ↓
Deploy
```

PIPELINE TASK 3 — FilePipeline

Create:

```
FilePipeline
```

Use the macOS/Linux version:

```
pipeline {
    agent any

    stages {
        stage('Create File') {
            steps {
                writeFile(
                    file: 'pipeline.txt',
                    text: 'Created by Jenkins Pipeline'
                )
            }
        }

        stage('Show File') {
            steps {
                sh 'cat pipeline.txt'
            }
        }
    }
}
```

Run it.

PIPELINE TASK 4 — UserPipeline

Create:

```
UserPipeline
```

Use:

```
pipeline {
    agent any

    parameters {
        string(
            name: 'USERNAME',
            defaultValue: 'User',
            description: 'Enter your name'
        )
    }

    stages {
        stage('Greet') {
            steps {
                echo "Hello, ${params.USERNAME}!"
            }
        }
    }
}
```

Save.

Choose:

Build with Parameters

Try different values.

PIPELINE TASK 5 — ScheduledPipeline

Create:

```
ScheduledPipeline
```

Use:

```
pipeline {
    agent any

    triggers {
        cron('H/5 * * * *')
    }

    stages {
        stage('Show Time') {
            steps {
                sh 'date'
            }
        }
    }
}
```

Verify multiple builds.

Then disable the trigger after testing.

PIPELINE TASK 6 — GitPipeline

Create GitHub repository:

```
jenkins-pipeline-git-demo
```

Add an `index.html`.

Then create Jenkins:

```
GitPipeline
```

Use:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/YOUR_USERNAME/jenkins-pipeline-gi
            }
        }

        stage('Build') {
            steps {
                echo 'Compiling code...'
            }
        }

        stage('Commit Info') {
            steps {
                sh 'git log -1 --pretty=%B'
            }
        }
    }
}
```

Run it.

You should see:

```
Checkout
Build
Commit Info
```

and the latest Git commit message.

PART 15 — Automatic Build #1: Poll SCM

Create GitHub repository:

```
jenkins-polling-demo
```

Add:

```
index.html
```

Create Jenkins:

```
Jenkins-Polling-Demo
```

Freestyle.

Source Code Management

Select:

Git

Repository:

```
https://github.com/YOUR_USERNAME/jenkins-polling-demo.git
```

Branch:

```
*/main
```

Build Triggers

Select:

Poll SCM

Schedule:

```
* * * * *
```

Build Step

Execute shell:

```
echo "Building Jenkins Application"
echo "Build Number: $BUILD_NUMBER"
echo "Workspace: $WORKSPACE"
```

Save.

Build manually once.

Then modify `index.html` and push:

```
git add .
git commit -m "Updated application"
git push origin main
```

Wait approximately one minute.

Jenkins should detect the change and create:

```
Build #2
```

PART 16 — Automatic Build #2: GitHub Webhook

Create:

```
jenkins-webhook-demo
```

with:

```
index.html
```

Create Jenkins:

```
Jenkins-Webhook-Demo
```

Freestyle.

Configure Git:

```
https://github.com/YOUR_USERNAME/jenkins-webhook-demo.git
```

Branch:

```
*/main
```

Build Trigger

Enable:

GitHub hook trigger for GITScm polling

Build Step

```
echo "=========================="
echo "Jenkins Automatic Build"
echo "=========================="
echo "Build Number: $BUILD_NUMBER"
echo "Build completed successfully."
```

Save.

Build manually once.

Important Mac issue

Your Jenkins is running at:

```
localhost:8080
```

GitHub cannot normally send a webhook to:

```
http://localhost:8080
```

because `localhost` means your Mac, not GitHub's servers.

Therefore, you need a **publicly reachable Jenkins URL** or an institution-approved tunneling setup.

Do not attempt the webhook portion until your Jenkins endpoint is reachable from outside your Mac.

PART 17 — GitHub webhook

Your public Jenkins address will look conceptually like:

```
https://YOUR-PUBLIC-JENKINS-URL
```

The webhook endpoint is:

```
https://YOUR-PUBLIC-JENKINS-URL/github-webhook/
```

In GitHub:

Repository → Settings → Webhooks → Add webhook

Payload URL:

```
https://YOUR-PUBLIC-JENKINS-URL/github-webhook/
```

Content type:

```
application/json
```

Choose:

Just the push event

Add webhook.

Then modify your application:

```
git add .
git commit -m "Updated application"
git push origin main
```

GitHub should send the webhook.

Jenkins should automatically create a new build.

PART 18 — Automatic Pipeline

Create repository:

```
jenkins-pipeline-demo
```

It should contain:

```
index.html
Jenkinsfile
```

Create `Jenkinsfile`:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out source code'
            }
        }

        stage('Build') {
            steps {
                echo 'Building application'
            }
        }

        stage('Test') {
            steps {
                echo 'Running test cases'
            }
        }

        stage('Package') {
            steps {
                echo 'Packaging application'
            }
        }
    }

    post {
        success {
            echo 'BUILD SUCCESSFUL'
        }

        failure {
            echo 'BUILD FAILED'
        }
    }
}
```

Commit:

```
git add .
git commit -m "Add Jenkins pipeline"
git push origin main
```

Jenkins configuration

Create:

```
Jenkins-Pipeline-Demo
```

Select:

Pipeline

Under Pipeline:

Definition: Pipeline script from SCM

SCM:

Git

Repository:

```
https://github.com/YOUR_USERNAME/jenkins-pipeline-demo.git
```

Branch:

```
*/main
```

Script Path:

```
Jenkinsfile
```

Click:

Build Now

Expected:

```
Checkout
   ↓
Build
   ↓
Test
   ↓
Package
   ↓
BUILD SUCCESSFUL
```

Then configure:

GitHub hook trigger for GITScm polling

and the GitHub webhook.

Change the application and push.

Jenkins should automatically execute the pipeline.

PART 19 — Build Number / Version task

Create repository:

```
jenkins-version-demo
```

Add:

```
index.html
```

Create Jenkins Freestyle project:

```
Jenkins-Version-Demo
```

Connect it to GitHub.

Build trigger

Enable:

```
GitHub hook trigger for GITScm polling
```

Build step

On macOS:

```
echo "================================"
echo "Jenkins Application Build"
echo "================================"
echo "Build Number: $BUILD_NUMBER"
echo "Job Name: $JOB_NAME"
echo "Build ID: $BUILD_ID"
echo "================================"
```

Demonstrate the build numbers

First build:

```
#1
```

Change application → push:

```
#2
```

Change application again → push:

```
#3
```

You should be able to demonstrate that each code change creates a new Jenkins build number.

PART 20 — Artifacts

Create a Freestyle job:

```
ArtifactJob
```

Build step:

```
echo "This is my Jenkins artifact" > artifact.txt
```

Then:

Post-build Actions

→ **Archive the artifacts**

Files to archive:

```
artifact.txt
```

Save → Build Now.

After the build, Jenkins should show the archived artifact.

PART 21 — Artifact retention

Go into:

Configure → General

Enable:

Discard old builds

Then configure:

```
Days to keep builds
```

or:

```
Max # of builds to keep
```

PART 22 — Fingerprints

Explore:

See Fingerprints

You should understand that Jenkins can track artifacts through their fingerprint/checksum information and identify where they have been used.

PART 23 — Jenkins security

Go to:

Manage Jenkins → Security

Depending on your Jenkins version this may be called:

Security

or:

Configure Global Security

Make sure Jenkins authentication is enabled.

The teacher's material describes:

```
Jenkins' own user database
```

and:

```
Matrix-based Security
```

with appropriate administrator permissions.

Important

Don't experiment with security settings until your practicals are working.

Otherwise you can accidentally lock yourself out.

PART 24 — Agents and Executors

Modern Jenkins terminology is:

```
Controller
Agent
```

The controller handles Jenkins administration while agents execute builds.

For your current Mac practical, you can use the local Jenkins executor.

You do **not** need a second computer just to complete the basic tasks.

PART 25 — Understand Executors

An executor determines how many builds a node can execute simultaneously.

You can explore:

Manage Jenkins → Nodes / Agents

and inspect the built-in node.

Don't change the configuration unless your teacher specifically asks you to.

PART 26 — Understand triggers

You should now be able to explain these three.

Build periodically

```
Jenkins
   ↓
Time arrives
   ↓
Build
```

Example:

```
H/2 * * * *
```

Poll SCM

```
Jenkins
   ↓
Check GitHub
   ↓
Did code change?
   ↓
Yes → Build
```

Webhook

```
Developer
   ↓
git push
   ↓
GitHub
   ↓
Webhook
   ↓
Jenkins
   ↓
Build
```

PART 27 — Final recommended completion order

Phase 1 — Installation

- [ ] Install Homebrew
- [ ] Install JDK 21
- [ ] Configure `JAVA_HOME`
- [ ] Verify Java
- [ ] Install Git
- [ ] Configure Git
- [ ] Install Jenkins
- [ ] Start Jenkins
- [ ] Open `localhost:8080`
- [ ] Unlock Jenkins
- [ ] Install suggested plugins
- [ ] Create administrator
- [ ] Verify dashboard

Phase 2 — Jenkins basics

- [ ] Explore Dashboard
- [ ] Explore Manage Jenkins

- [ ] Explore Plugins
- [ ] Explore Credentials
- [ ] Understand Jobs
- [ ] Understand Builds
- [ ] Understand Workspace
- [ ] Understand Executors
- [ ] Understand environment variables

Phase 3 — Freestyle

- [ ] `HelloWorldJob`
- [ ] `WorkspaceJob`
- [ ] `FileCopyJob`
- [ ] `ParameterizedJob`
- [ ] `ScheduledJob`
- [ ] `GitJob`

Phase 4 — Pipeline

- [ ] `EchoPipeline`
- [ ] `BuildTestPipeline`
- [ ] `FilePipeline`
- [ ] `UserPipeline`
- [ ] `ScheduledPipeline`
- [ ] `GitPipeline`

Phase 5 — Automatic builds

- [ ] `Jenkins-Polling-Demo`
- [ ] Test SCM polling
- [ ] `Jenkins-Webhook-Demo`
- [ ] Configure GitHub webhook
- [ ] Test webhook
- [ ] `Jenkins-Pipeline-Demo`
- [ ] Create Jenkinsfile
- [ ] Configure Pipeline from SCM
- [ ] Configure webhook
- [ ] Test automatic Pipeline
- [ ] `Jenkins-Version-Demo`
- [ ] Demonstrate Build #1
- [ ] Demonstrate Build #2
- [ ] Demonstrate Build #3

Phase 6 — Advanced Jenkins topics

- [ ] Archive artifact
- [ ] Artifact retention

- [ ] Fingerprints
- [ ] Security
- [ ] Credentials
- [ ] Executors
- [ ] Agents
- [ ] Upstream/downstream concepts
- [ ] Build triggers
- [ ] GitHub hooks

Windows → macOS command translation

Teacher's Windows	Your macOS
Execute Windows batch command	Execute shell
`%WORKSPACE%`	`$WORKSPACE`
`%BUILD_NUMBER%`	`$BUILD_NUMBER`
`%JOB_NAME%`	`$JOB_NAME`
`%BUILD_ID%`	`$BUILD_ID`
`%USERNAME%`	`$USERNAME`
`%date% %time%`	`$(date)`
`echo ... > C:\...`	`echo ... > "$WORKSPACE/..."`
`copy`	`cp`
PowerShell	`sh`
`Get-Date`	`date`
`Get-Content`	`cat`
`C:\...`	`/Users/...`

Start here

Don't try to complete everything at once.

Open Terminal and run these three commands:

```
brew --version
```

```
java -version
```

```
git --version
```

Send me the output of those three commands.

Then proceed in order:

JDK → Jenkins installation → first login → plugins → first Freestyle job.

We should verify each stage before moving to the next one.